Financial Econometrics Problem Set 3

Francesco Zucca, Michael Fehl, Giuseppe Lista

2026-03-07

Introduction

In this practice, we will work on fitting different volatility models (ARCH and GARCH). We will also focus on discussing the validation and selection of the model. As done in previous practices, each student group must follow the practice script below, performing the necessary calculations and graphs in the R environment. The obtained results and answers to the questions should be compiled into a document containing your solution, created, for example, in R Markdown or Microsoft Word. Please make sure to include your full names and group number at the beginning of the document. Submit the questionnaire solution in PDF format by uploading it to the platform under the corresponding task section before the deadline.

Questionnaire

In this questionnaire, we will work with the following time series:

- KO: The Coca-Cola Company (KO) stock (NYSE)

You must:

1. Download the time series. You may work with the complete dataset (from 2007 to the most recent trading session).
2. Create a graphical representation of the time series using the Adjusted Closing Price.
3. Compute the log-returns of the series. It is not necessary to model the mean (e.g., using an ARIMA model).
4. Using the fGarch package, fit the following models to the log-returns: an ARCH(1) model, an ARCH(3) model, and a GARCH(1,1) model.
5. Write the expression of each fitted model.
6. Analyze the diagnostic results and determine which of the three models provides the best fit.

Setup

We load the necessary libraries.

```r
library(quantmod)
library(fGarch)
library(fBasics)
```

1 - Downloading the time series

Instructions: download the time series. You may work with the complete dataset (from 2007 to the most recent trading session).

We download the time series and load the complete dataset.

```r
getSymbols("KO") # download the time series
```

```
## [1] "KO"
```

```r
val = get("KO")
```

We display the first and last observations of the time series.

```r
head(val)
```

```
##            KO.Open KO.High KO.Low KO.Close KO.Volume KO.Adjusted
## 2007-01-03  24.180  24.440 24.140   24.290  15753400    13.50413
## 2007-01-04  24.210  24.350 24.125   24.300  11810400    13.50969
## 2007-01-05  24.250  24.285 24.085   24.130  11607000    13.41518
## 2007-01-08  24.005  24.335 24.005   24.285  17551000    13.50136
## 2007-01-09  24.270  24.415 24.230   24.305  13724000    13.51247
## 2007-01-10  24.245  24.395 24.185   24.340   8928000    13.53193
```

```r
tail(val)
```

```
##            KO.Open KO.High KO.Low KO.Close KO.Volume KO.Adjusted
## 2026-03-09   76.62  78.070  76.53    77.80  18554100    77.26871
## 2026-03-10   77.48  78.410  77.10    77.88  17243900    77.34815
## 2026-03-11   77.71  77.710  76.60    77.63  11546800    77.09986
## 2026-03-12   77.42  78.160  76.90    77.61  16842200    77.08000
## 2026-03-13   77.47  78.050  77.19    77.34  11902700    77.34000
## 2026-03-13   77.47  78.045  77.19    77.34   9399025    77.34000
```

2 - Graphical representation

Instructions: create a graphical representation of the time series using the Adjusted Closing Price.

First, we retrieve the time series of the adjusted closing prices.

```r
series <- val[ , 6]
```

Now we can plot the adjusted closing prices time series.

```r
plot(series)
```

3 - Computing log-returns

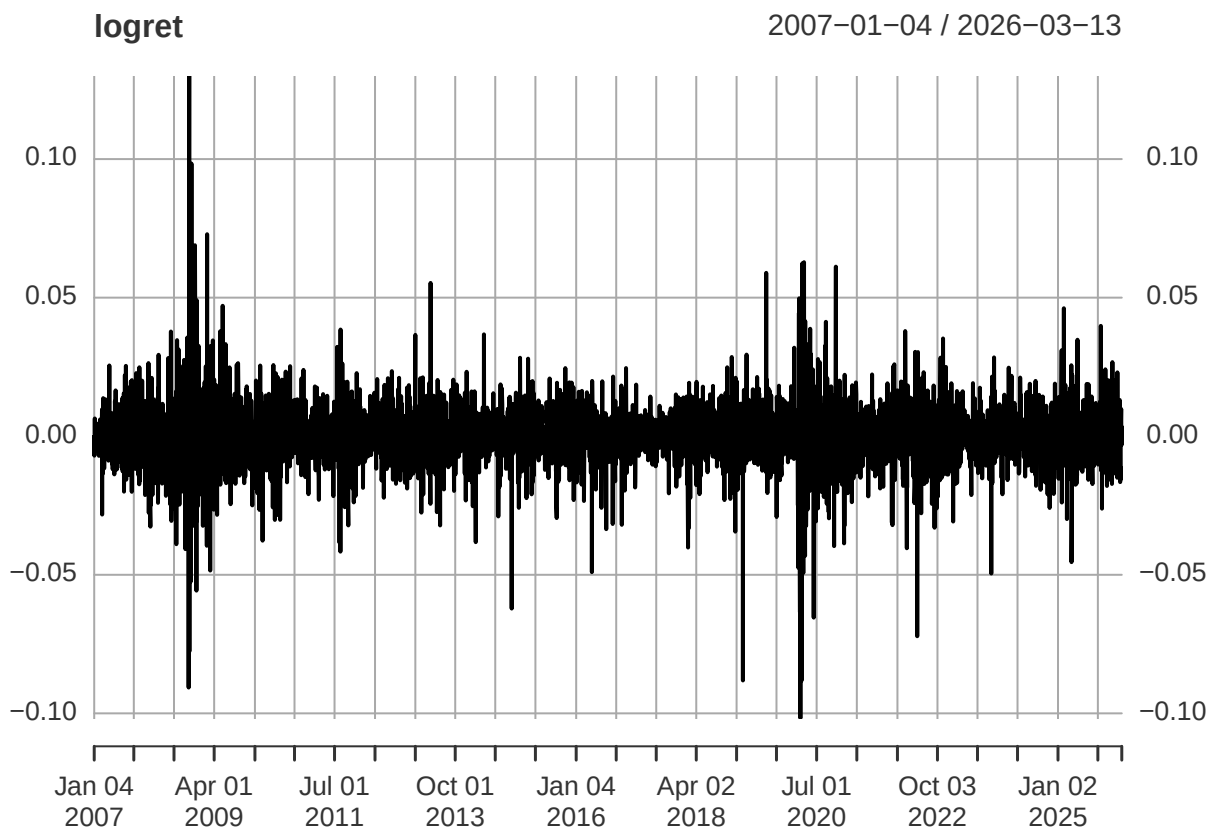
Instructions: compute the log-returns of the series. It is not necessary to model the mean (e.g., using an ARIMA model).

We compute the log returns of the time series.

```r
lnseries <- log(series) # log transformation
logret <- diff(lnseries)[-1] # compute log-returns
```

Now we plot the log-returns.

```r
plot(logret)
```



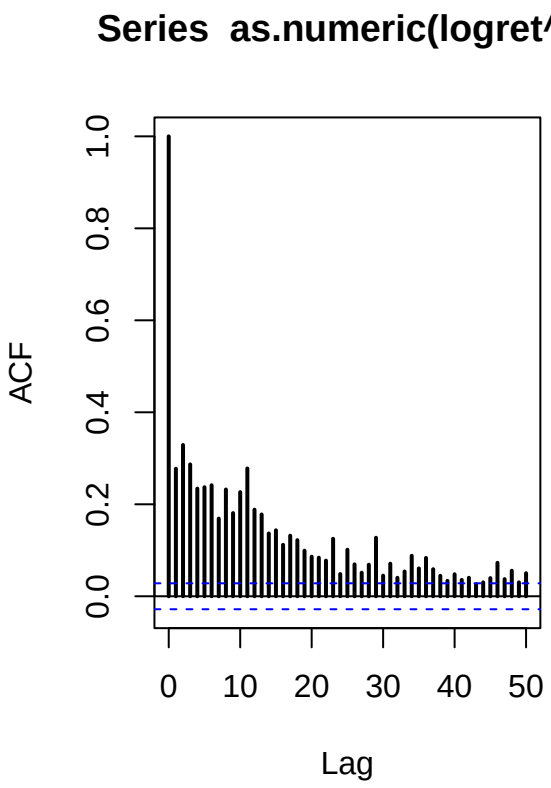
Before moving on to model fitting, we do some basic analysis of the returns.

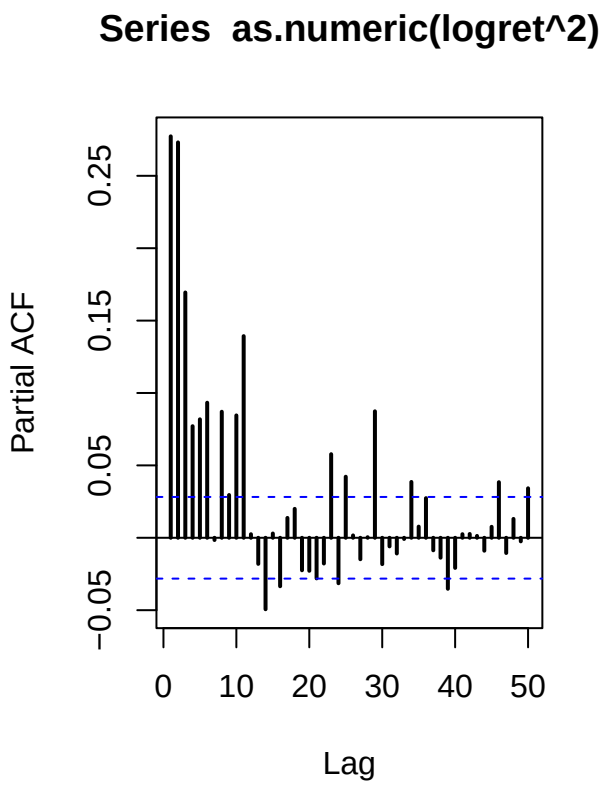
```
stats <- basicStats(logret)
stats[c("Skewness", "Kurtosis"), , drop = FALSE]
```

```
##          KO.Adjusted
## Skewness   -0.161362
## Kurtosis   11.313256
```

The time series is more skewed compared to the normal distribution, and it has fatter tails because of the excess kurtosis of around 11.

```
par(mfrow = c(1, 2))
acf(as.numeric(logret^2), lwd = 2, lag.max = 50)
pacf(as.numeric(logret^2), lwd = 2, lag.max = 50)
```






```
par(mfrow = c(1, 1))
```

Looking at the ACF and PACF of the squared log-returns, a lot of lags cross the confidence intervals, suggesting dependence among squared log-returns, and therefore non-constant variance. Hence, we should indeed apply volatility models.

4 and 5 - Fitting volatility models

Instructions: using the fGarch package, fit the following models to the log-returns: an ARCH(1) model, an ARCH(3) model, and a GARCH(1,1) model. Write the expression of each fitted model.

ARCH(1)

First, we fit an $ARCH(1)$ model to our time series.

```
arch1 <- garchFit(~ garch(1, 0), data = logret)
```

```
##
## Series Initialization:
##  ARMA Model:                arma
##  Formula Mean:              ~ arma(0, 0)
```

```
##  GARCH Model:               garch
##  Formula Variance:          ~ garch(1, 0)
##  ARMA Order:                0 0
##  Max ARMA Order:            0
##  GARCH Order:               1 0
##  Max GARCH Order:           1
##  Maximum Order:             1
##  Conditional Dist:          norm
##  h.start:                   2
##  llh.start:                 1
##  Length of Series:          4829
##  Recursion Init:            mci
##  Series Scale:              0.01175243
##
## Parameter Initialization:
##  Initial Parameters:          $params
##  Limits of Transformations:   $U, $V
##  Which Parameters are Fixed?  $includes
##  Parameter Matrix:
##                       U           V     params includes
##     mu     -0.30751352   0.3075135 0.03075135     TRUE
##     omega   0.00000100 100.0000000 0.10000000     TRUE
##     alpha1  0.00000001   1.0000000 0.10000000     TRUE
##     gamma1 -0.99999999   1.0000000 0.10000000    FALSE
##     delta   0.00000000   2.0000000 2.00000000    FALSE
##     skew    0.10000000  10.0000000 1.00000000    FALSE
##     shape   1.00000000  10.0000000 4.00000000    FALSE
##  Index List of Parameters to be Optimized:
##     mu  omega alpha1
##      1      2      3
##  Persistence:                  0.1
##
##
## --- START OF TRACE ---
## Selected Algorithm: nlminb
##
## R coded nlminb Solver:
##
##   0:     13687.572: 0.0307514 0.100000 0.100000
##   1:     6672.9520: 0.0307588  1.06230 0.372004
##   2:     6665.4991: 0.0308203  1.06557 0.331807
##   3:     6503.4641: 0.0308591 0.759277 0.229173
##   4:     6485.5652: 0.0309092 0.666617 0.298107
##   5:     6484.4527: 0.0309817 0.669437 0.339063
##   6:     6483.9456: 0.0321511 0.654191 0.335548
##   7:     6482.6807: 0.0463110 0.655992 0.338373
##   8:     6482.5483: 0.0524908 0.653990 0.341544
##   9:     6482.5481: 0.0524532 0.654295 0.341447
##  10:     6482.5481: 0.0524528 0.654281 0.341441
##
## Final Estimate of the Negative LLH:
##  LLH:  -14976.06    norm LLH:  -3.101275
##          mu       omega      alpha1
## 0.000616448 0.000090369 0.341440671
```

```
##
## R-optimhess Difference Approximated Hessian Matrix:
##                   mu         omega       alpha1
## mu     -48074592.094     -17924440     2507.698
## omega  -17924439.787 -208927404043 -8083000.329
## alpha1      2507.698      -8083000    -1797.012
## attr(,"time")
## Time difference of 0.008251905 secs
##
## --- END OF TRACE ---
##
##
## Time to Estimate Parameters:
##  Time difference of 0.02451992 secs
```

```
summary(arch1)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 0), data = logret)
##
## Mean and Variance Equation:
##  data ~ garch(1, 0)
## <environment: 0x1199229e8>
##  [data = logret]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##         mu       omega      alpha1
## 6.1645e-04  9.0369e-05  3.4144e-01
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     6.164e-04   1.442e-04    4.274 1.92e-05 ***
## omega  9.037e-05   2.407e-06   37.539  < 2e-16 ***
## alpha1 3.414e-01   2.596e-02   13.154  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  14976.06    normalized:  3.101275
##
## Description:
##  Sat Mar 14 19:42:21 2026 by user:
##
##
```

```
##
## Standardised Residuals Tests:
##                                 Statistic    p-Value
##  Jarque-Bera Test   R    Chi^2  7968.975664 0.00000000
##  Shapiro-Wilk Test  R    W         0.943984 0.00000000
##  Ljung-Box Test     R    Q(10)    22.879001 0.01120212
##  Ljung-Box Test     R    Q(15)    24.364302 0.05915965
##  Ljung-Box Test     R    Q(20)    30.424804 0.06325654
##  Ljung-Box Test     R^2  Q(10)   230.560729 0.00000000
##  Ljung-Box Test     R^2  Q(15)   337.567608 0.00000000
##  Ljung-Box Test     R^2  Q(20)   407.699730 0.00000000
##  LM Arch Test       R    TR^2    212.249147 0.00000000
##
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.201308 -6.197280 -6.201308 -6.199894
```

Notice that every estimated coefficient is statistically significant.

The expression of an $ARCH(1)$ model is (setting the mean of the returns equal to 0 by assumption):

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2$$

The coefficients estimated by the model are the following:

```
cat("ARCH(1) fitted coefficients:\n")
```

```
## ARCH(1) fitted coefficients:
```

```
print(round(arch1@fit$coef, 6))
```

```
##       mu    omega   alpha1
## 0.000616 0.000090 0.341441
```

Substituting them into the model expression this model we obtain:

$$\sigma_t^2 = 0.000091 + 0.341109 \times r_{t-1}^2$$

ARCH(3)

We fit an $ARCH(3)$ model to our time series.

```
arch3 <- garchFit(~ garch(3, 0), data = logret)
```

```
##
## Series Initialization:
##  ARMA Model:                arma
##  Formula Mean:              ~ arma(0, 0)
##  GARCH Model:               garch
##  Formula Variance:          ~ garch(3, 0)
```

```
##  ARMA Order:                 0 0
##  Max ARMA Order:             0
##  GARCH Order:                3 0
##  Max GARCH Order:            3
##  Maximum Order:              3
##  Conditional Dist:           norm
##  h.start:                    4
##  llh.start:                  1
##  Length of Series:           4829
##  Recursion Init:             mci
##  Series Scale:               0.01175243
##
## Parameter Initialization:
##  Initial Parameters:          $params
##  Limits of Transformations:   $U, $V
##  Which Parameters are Fixed?  $includes
##  Parameter Matrix:
##                        U           V     params includes
##     mu     -0.30751352   0.3075135 0.03075135     TRUE
##     omega   0.00000100 100.0000000 0.10000000     TRUE
##     alpha1  0.00000001   1.0000000 0.03333333     TRUE
##     alpha2  0.00000001   1.0000000 0.03333333     TRUE
##     alpha3  0.00000001   1.0000000 0.03333333     TRUE
##     gamma1 -0.99999999   1.0000000 0.10000000    FALSE
##     gamma2 -0.99999999   1.0000000 0.10000000    FALSE
##     gamma3 -0.99999999   1.0000000 0.10000000    FALSE
##     delta   0.00000000   2.0000000 2.00000000    FALSE
##     skew    0.10000000  10.0000000 1.00000000    FALSE
##     shape   1.00000000  10.0000000 4.00000000    FALSE
##  Index List of Parameters to be Optimized:
##     mu  omega alpha1 alpha2 alpha3
##      1      2      3      4      5
##  Persistence:                  0.1
##
##
## --- START OF TRACE ---
## Selected Algorithm: nlminb
##
## R coded nlminb Solver:
##
##   0:     11630.372: 0.0307514 0.100000 0.0333333 0.0333333 0.0333333
##   1:     6765.2901: 0.0307557 0.860396 0.392670 0.426706 0.404725
##   2:     6716.9791: 0.0650930 0.914266 1.00000e-08  1.00000 1.00000e-08
##   3:     6537.9936: 0.124493 0.406477 1.00000e-08  1.00000 0.104355
##   4:     6536.3906: 0.132942 0.423042 0.328822 0.911603 0.0901347
##   5:     6460.2773: 0.135625 0.363830 0.244075 0.741882 0.0606107
##   6:     6405.2683: 0.134557 0.485225 0.159929 0.610188 0.147842
##   7:     6364.9047: 0.134428 0.419725 0.105951 0.538651 0.101540
##   8:     6341.4186: 0.134186 0.488403 0.120272 0.446372 0.132910
##   9:     6338.0683: 0.131067 0.450444 0.0299131 0.251869 0.118621
##  10:     6300.0764: 0.131002 0.505138 0.131525 0.261274 0.151387
##  11:     6293.5858: 0.133735 0.476201 0.135861 0.186629 0.138914
##  12:     6291.8184: 0.130386 0.512527 0.118026 0.168131 0.164226
##  13:     6288.8000: 0.126786 0.494613 0.113555 0.182566 0.149069
```

```
##  14:     6288.4615: 0.123269 0.501434 0.140640 0.207399 0.149041
##  15:     6284.5469: 0.119653 0.489117 0.124283 0.193350 0.142729
##  16:     6282.7517: 0.115986 0.499792 0.125331 0.185857 0.152580
##  17:     6280.9019: 0.112312 0.490646 0.117608 0.189527 0.145479
##  18:     6274.8029: 0.0948809 0.486160 0.137300 0.219190 0.137225
##  19:     6272.1905: 0.0873396 0.477258 0.147481 0.165198 0.140396
##  20:     6270.1386: 0.0797767 0.507683 0.112900 0.179125 0.161526
##  21:     6268.5281: 0.0720726 0.492269 0.108735 0.167560 0.151777
##  22:     6267.2919: 0.0645159 0.485240 0.161512 0.170621 0.147690
##  23:     6265.2294: 0.0486262 0.494049 0.138073 0.184284 0.154713
##  24:     6265.1547: 0.0486223 0.478713 0.126713 0.178766 0.144123
##  25:     6264.6431: 0.0486203 0.486412 0.131360 0.183962 0.148463
##  26:     6264.6261: 0.0482941 0.485301 0.129850 0.186635 0.146577
##  27:     6264.6185: 0.0479489 0.485962 0.130184 0.186828 0.146835
##  28:     6264.6094: 0.0465645 0.486021 0.130200 0.185920 0.146680
##  29:     6264.6074: 0.0462852 0.485531 0.130146 0.186909 0.147211
##  30:     6264.6074: 0.0460833 0.486396 0.129661 0.187628 0.146318
##  31:     6264.6067: 0.0461735 0.485988 0.129848 0.187235 0.146800
##  32:     6264.6067: 0.0461730 0.485972 0.129881 0.187246 0.146776
##  33:     6264.6067: 0.0461731 0.485979 0.129868 0.187242 0.146784
##
## Final Estimate of the Negative LLH:
##  LLH:  -15194    norm LLH:  -3.146407
##           mu        omega       alpha1       alpha2       alpha3
## 5.426458e-04 6.712315e-05 1.298683e-01 1.872422e-01 1.467841e-01
##
## R-optimhess Difference Approximated Hessian Matrix:
##                   mu         omega        alpha1        alpha2        alpha3
## mu     -53377579.045      11828873      -338.264     -4670.650      4557.967
## omega   11828872.927 -274603769385 -13468381.765 -11941847.829 -12749281.741
## alpha1      -338.264     -13468382     -3689.042     -1139.651     -1052.348
## alpha2     -4670.650     -11941848     -1139.651     -2726.098     -1151.901
## alpha3      4557.967     -12749282     -1052.348     -1151.901     -3455.111
## attr(,"time")
## Time difference of 0.02119303 secs
##
## --- END OF TRACE ---
##
##
## Time to Estimate Parameters:
##  Time difference of 0.09510612 secs
```

```
summary(arch3)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(3, 0), data = logret)
##
## Mean and Variance Equation:
##  data ~ garch(3, 0)
## <environment: 0x119b5ce48>
```

```
##  [data = logret]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##         mu       omega      alpha1      alpha2      alpha3
## 5.4265e-04  6.7123e-05  1.2987e-01  1.8724e-01  1.4678e-01
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     5.426e-04   1.369e-04    3.964 7.38e-05 ***
## omega  6.712e-05   2.315e-06   28.990  < 2e-16 ***
## alpha1 1.299e-01   1.870e-02    6.946 3.75e-12 ***
## alpha2 1.872e-01   2.231e-02    8.392  < 2e-16 ***
## alpha3 1.468e-01   1.934e-02    7.591 3.18e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  15194    normalized:  3.146407
##
## Description:
##  Sat Mar 14 19:42:21 2026 by user:
##
##
##
## Standardised Residuals Tests:
##                                   Statistic    p-Value
##  Jarque-Bera Test   R    Chi^2  6806.2521997 0.00000000
##  Shapiro-Wilk Test  R    W         0.9585433 0.00000000
##  Ljung-Box Test     R    Q(10)    18.3627127 0.04914485
##  Ljung-Box Test     R    Q(15)    21.4357256 0.12347583
##  Ljung-Box Test     R    Q(20)    26.7566074 0.14225158
##  Ljung-Box Test     R^2  Q(10)    14.5501114 0.14934501
##  Ljung-Box Test     R^2  Q(15)    25.2745102 0.04637608
##  Ljung-Box Test     R^2  Q(20)    36.2678168 0.01430289
##  LM Arch Test       R    TR^2     19.5030538 0.07709004
##
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.290743 -6.284031 -6.290745 -6.288386
```

Again, all coefficients are statistically significant for $ARCH(3)$.

The expression of an $ARCH(3)$ model is (setting the mean of the returns equal to 0 by assumption):

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2 + \alpha_2 r_{t-2}^2 + \alpha_3 r_{t-3}^2$$

The coefficients estimated by the model are the following:

```r
cat("ARCH(3) fitted coefficients:\n")
```

```
## ARCH(3) fitted coefficients:
```

```r
print(round(arch3@fit$coef, 6))
```

```
##       mu    omega   alpha1   alpha2   alpha3
## 0.000543 0.000067 0.129868 0.187242 0.146784
```

Substituting them into the model expression this model we obtain:

$$\sigma_t^2 = 0.000067 + 0.129557 \times r_{t-1}^2 + 0.187017 \times r_{t-2}^2 + 0.147060 \times r_{t-3}^2$$

GARCH(1,1)

We fit an $GARCH(1,1)$ model to our time series.

```r
garch11 <- garchFit(~ garch(1, 1), data = logret)
```

```
##
## Series Initialization:
##  ARMA Model:                arma
##  Formula Mean:              ~ arma(0, 0)
##  GARCH Model:               garch
##  Formula Variance:          ~ garch(1, 1)
##  ARMA Order:                0 0
##  Max ARMA Order:            0
##  GARCH Order:               1 1
##  Max GARCH Order:           1
##  Maximum Order:             1
##  Conditional Dist:          norm
##  h.start:                   2
##  llh.start:                 1
##  Length of Series:          4829
##  Recursion Init:            mci
##  Series Scale:              0.01175243
##
## Parameter Initialization:
##  Initial Parameters:          $params
##  Limits of Transformations:   $U, $V
##  Which Parameters are Fixed?  $includes
##  Parameter Matrix:
##                      U           V     params includes
##     mu     -0.30751352   0.3075135 0.03075135     TRUE
##     omega   0.00000100 100.0000000 0.10000000     TRUE
##     alpha1  0.00000001   1.0000000 0.10000000     TRUE
##     gamma1 -0.99999999   1.0000000 0.10000000    FALSE
##     beta1   0.00000001   1.0000000 0.80000000     TRUE
##     delta   0.00000000   2.0000000 2.00000000    FALSE
##     skew    0.10000000  10.0000000 1.00000000    FALSE
```

```
##     shape   1.00000000  10.0000000 4.00000000    FALSE
##  Index List of Parameters to be Optimized:
##     mu  omega alpha1  beta1
##      1      2      3      5
##  Persistence:                  0.9
##
##
## --- START OF TRACE ---
## Selected Algorithm: nlminb
##
## R coded nlminb Solver:
##
##   0:     6225.5006: 0.0307514 0.100000 0.100000 0.800000
##   1:     6206.9752: 0.0307518 0.0893241 0.0972497 0.793678
##   2:     6200.9037: 0.0307533 0.0790332 0.104446 0.795630
##   3:     6199.8083: 0.0307543 0.0803134 0.109853 0.801331
##   4:     6196.3587: 0.0307568 0.0729872 0.107816 0.803686
##   5:     6192.5024: 0.0307629 0.0669308 0.106779 0.818372
##   6:     6181.2975: 0.0307884 0.0362312 0.0814382 0.872265
##   7:     6179.4663: 0.0307885 0.0380556 0.0826205 0.873856
##   8:     6177.5254: 0.0308336 0.0308518 0.0713567 0.892201
##   9:     6177.0697: 0.0308349 0.0288982 0.0740200 0.894068
##  10:     6177.0103: 0.0308350 0.0281458 0.0736913 0.893732
##  11:     6176.9010: 0.0308485 0.0281229 0.0737360 0.894501
##  12:     6176.8407: 0.0308688 0.0270854 0.0733978 0.895735
##  13:     6176.7318: 0.0309557 0.0265293 0.0730259 0.897777
##  14:     6176.6614: 0.0310602 0.0259665 0.0725077 0.898453
##  15:     6176.2534: 0.0334262 0.0224440 0.0666271 0.908603
##  16:     6176.1977: 0.0358208 0.0237515 0.0647329 0.907622
##  17:     6176.0579: 0.0382162 0.0238573 0.0660790 0.907118
##  18:     6176.0054: 0.0406117 0.0233402 0.0669372 0.906132
##  19:     6175.9489: 0.0430073 0.0235482 0.0677557 0.905855
##  20:     6175.9428: 0.0411629 0.0229679 0.0663122 0.907880
##  21:     6175.9377: 0.0423037 0.0231721 0.0665143 0.907287
##  22:     6175.9368: 0.0422063 0.0231103 0.0666899 0.907243
##  23:     6175.9366: 0.0421086 0.0231247 0.0666918 0.907243
##  24:     6175.9366: 0.0420789 0.0231188 0.0666788 0.907264
##  25:     6175.9366: 0.0420804 0.0231204 0.0666811 0.907260
##
## Final Estimate of the Negative LLH:
##  LLH:  -15282.67    norm LLH:  -3.164769
##           mu        omega       alpha1        beta1
## 4.945469e-04 3.193378e-06 6.668115e-02 9.072604e-01
##
## R-optimhess Difference Approximated Hessian Matrix:
##                    mu         omega        alpha1         beta1
## mu     -5.452786e+07 -6.830273e+07  1.984432e+04  4.063688e+02
## omega  -6.830273e+07 -3.781208e+13 -2.390040e+09 -3.300186e+09
## alpha1  1.984432e+04 -2.390040e+09 -2.264485e+05 -2.524697e+05
## beta1   4.063688e+02 -3.300186e+09 -2.524697e+05 -3.205295e+05
## attr(,"time")
## Time difference of 0.008491993 secs
##
## --- END OF TRACE ---
```

```
##
##
## Time to Estimate Parameters:
##  Time difference of 0.03979206 secs
```

```
summary(garch11)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~garch(1, 1), data = logret)
##
## Mean and Variance Equation:
##  data ~ garch(1, 1)
## <environment: 0x10f9185c0>
##  [data = logret]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##         mu       omega      alpha1       beta1
## 4.9455e-04  3.1934e-06  6.6681e-02  9.0726e-01
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     4.945e-04   1.354e-04    3.651 0.000261 ***
## omega  3.193e-06   6.676e-07    4.783 1.72e-06 ***
## alpha1 6.668e-02   7.870e-03    8.472  < 2e-16 ***
## beta1  9.073e-01   1.199e-02   75.690  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  15282.67    normalized:  3.164769
##
## Description:
##  Sat Mar 14 19:42:21 2026 by user:
##
##
##
## Standardised Residuals Tests:
##                                   Statistic    p-Value
##  Jarque-Bera Test   R    Chi^2  6424.5365198 0.00000000
##  Shapiro-Wilk Test  R    W         0.9602602 0.00000000
##  Ljung-Box Test     R    Q(10)    17.9065230 0.05656109
##  Ljung-Box Test     R    Q(15)    19.5665716 0.18919730
##  Ljung-Box Test     R    Q(20)    23.9170511 0.24603433
##  Ljung-Box Test     R^2  Q(10)     4.0154535 0.94664715
```

```
##  Ljung-Box Test     R^2  Q(15)     4.5460340 0.99532217
##  Ljung-Box Test     R^2  Q(20)     6.0072957 0.99888762
##  LM Arch Test       R    TR^2      4.1348321 0.98087856
##
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.327881 -6.322511 -6.327882 -6.325996
```

Again, all estimated coefficients are statistically significant.

The expression of a $GARCH(1,1)$ model is (setting the mean of the returns equal to 0 by assumption):

$$\sigma_t^2 = \omega + \alpha_1 r_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

The coefficients estimated by the model are the following:

```
cat("GARCH(1,1) fitted coefficients:\n")
```

```
## GARCH(1,1) fitted coefficients:
```

```
print(round(garch11@fit$coef, 6))
```

```
##       mu    omega   alpha1    beta1
## 0.000495 0.000003 0.066681 0.907260
```

Substituting them into the model expression this model we obtain:

$$\sigma_t^2 = 0.000003 + 0.066569 \times r_{t-1}^2 + 0.907679 \times \sigma_{t-1}^2$$

6 - Diagnostic results and model decision

Instructions: analyze the diagnostic results and determine which of the three models provides the best fit.

First, we compute the standardized residuals of each fitted model, defined as the residuals divided by the estimated conditional standard deviation:

$$\tilde{\varepsilon}_t = \varepsilon_t / \hat{\sigma}_t$$

Since we assume that the mean of the log-returns is equal to zero (we are not fitting a model for the mean), the log-returns are equal to the residuals themselves, i.e.:

$$\varepsilon_t = r_t$$

Hence, the standardized residuals simplify to:

$$\tilde{\varepsilon}_t = r_t / \hat{\sigma}_t$$

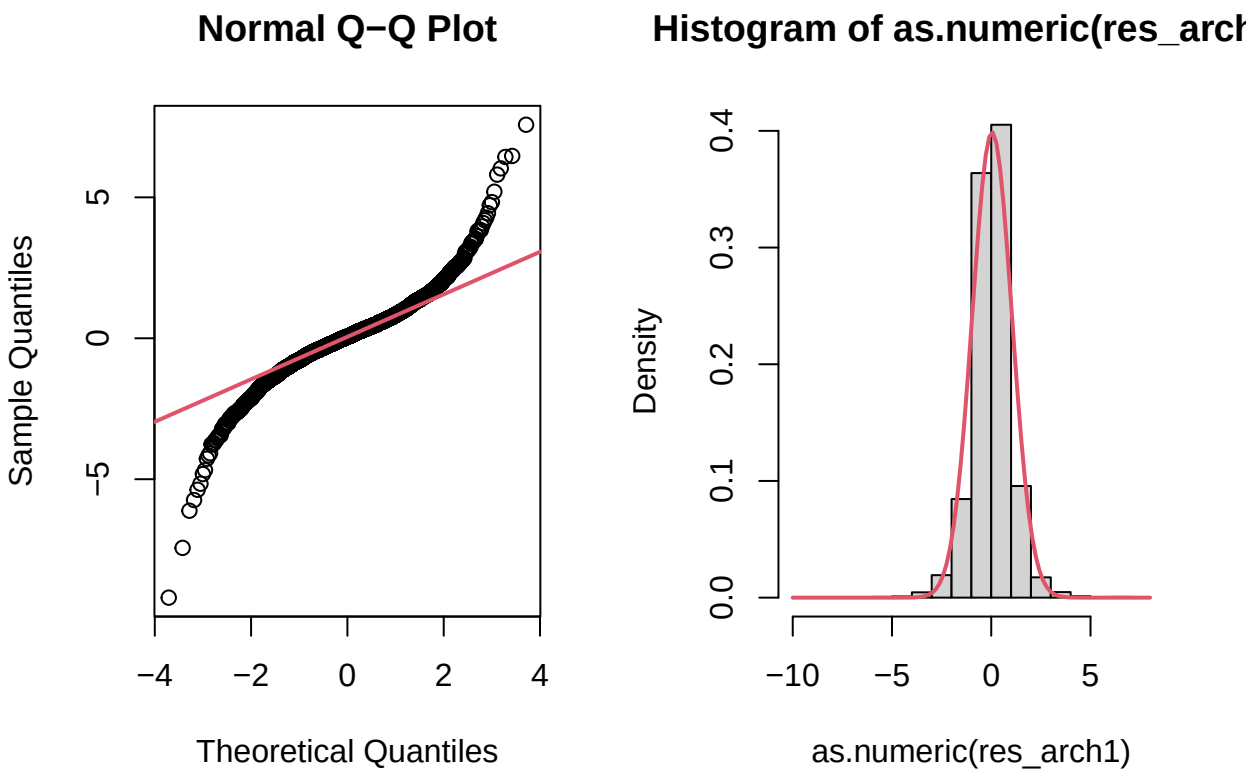
```
res_arch1 = logret / arch1@sigma.t
res_arch3 = logret / arch3@sigma.t
res_garch11 = logret / garch11@sigma.t
```

Testing normality on standardized residuals

Since all of these models assume the standardized residuals to be normally distributed, we plot their distribution against the one of the normal.

ARCH(1)

```
par(mfrow = c(1, 2))
qqnorm(as.numeric(res_arch1))
qqline(as.numeric(res_arch1), col = 2, lwd = 2)
hist(as.numeric(res_arch1), probability = TRUE)
curve(dnorm(x, mean = mean(as.numeric(res_arch1)), sd = sd(as.numeric(res_arch1))), lwd = 2, col = 2, a
```




```
par(mfrow = c(1, 1))
```

```
stats <- basicStats(as.numeric(res_arch1))
stats[c("Skewness", "Kurtosis"), , drop = FALSE]
```

```
##          as.numeric.res_arch1
## Skewness            -0.135751
## Kurtosis             6.272332
```

Both the distribution plots and the skewness and kurtosis of the distribution suggest that the residuals are not normally distributed.

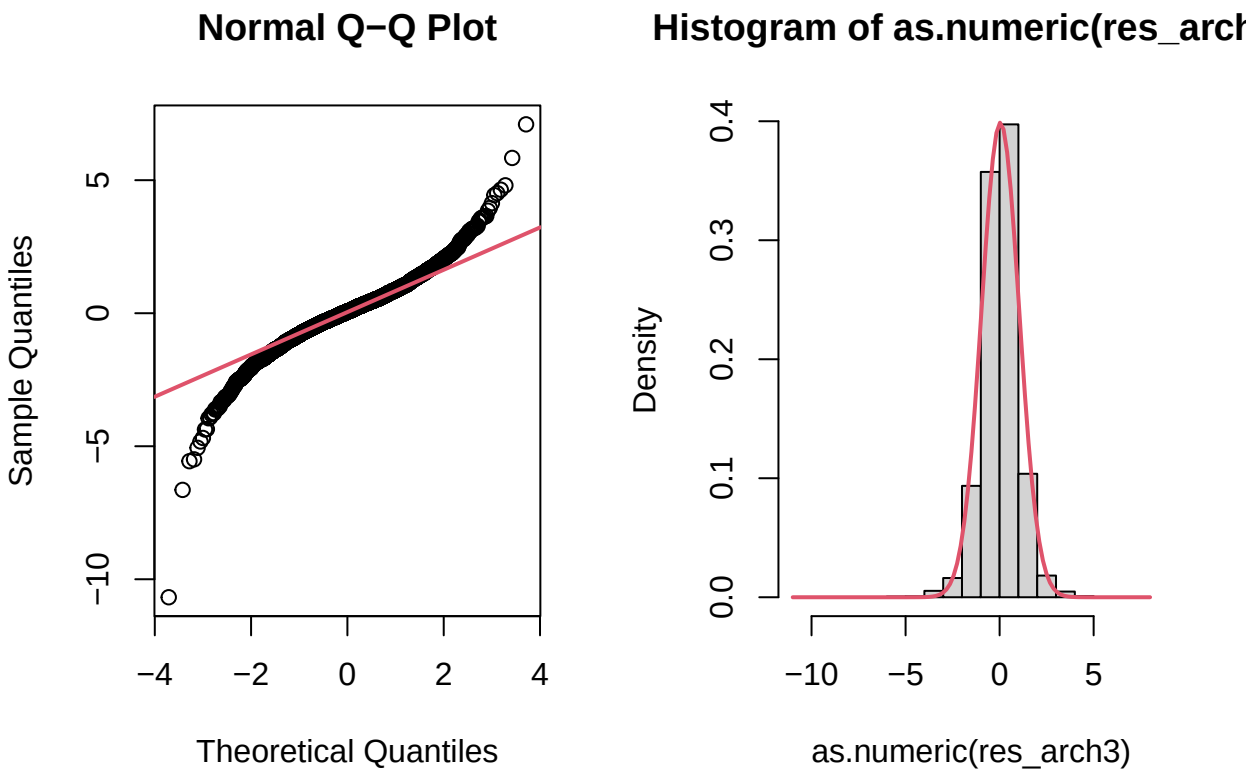
```
out <- capture.output(summary(arch1))
start <- which(grepl("Jarque-Bera", out))
end   <- which(grepl("Shapiro-Wilk", out))
cat(out[start:end], sep = "\n")
```

```
##  Jarque-Bera Test   R    Chi^2  7968.975664 0.00000000
##  Shapiro-Wilk Test  R    W         0.943984 0.00000000
```

This is further confirmed by the Jarque-Bera and Shapiro-Wilk tests, which both reported a p-value of 0.000, rejecting the null-hypothesis of normality.

ARCH(3)

```
par(mfrow = c(1, 2))
qqnorm(as.numeric(res_arch3))
qqline(as.numeric(res_arch3), col = 2, lwd = 2)
hist(as.numeric(res_arch3), probability = TRUE)
curve(dnorm(x, mean = mean(as.numeric(res_arch3)), sd = sd(as.numeric(res_arch3))), lwd = 2, col = 2, a
```



```r
par(mfrow = c(1, 1))
```

```r
stats <- basicStats(as.numeric(res_arch3))
stats[c("Skewness", "Kurtosis"), , drop = FALSE]
```

```
##          as.numeric.res_arch3
## Skewness            -0.377524
## Kurtosis             5.749683
```

Again, both the distribution plots and the skewness and kurtosis of the distribution suggest that the residuals are not normally distributed.

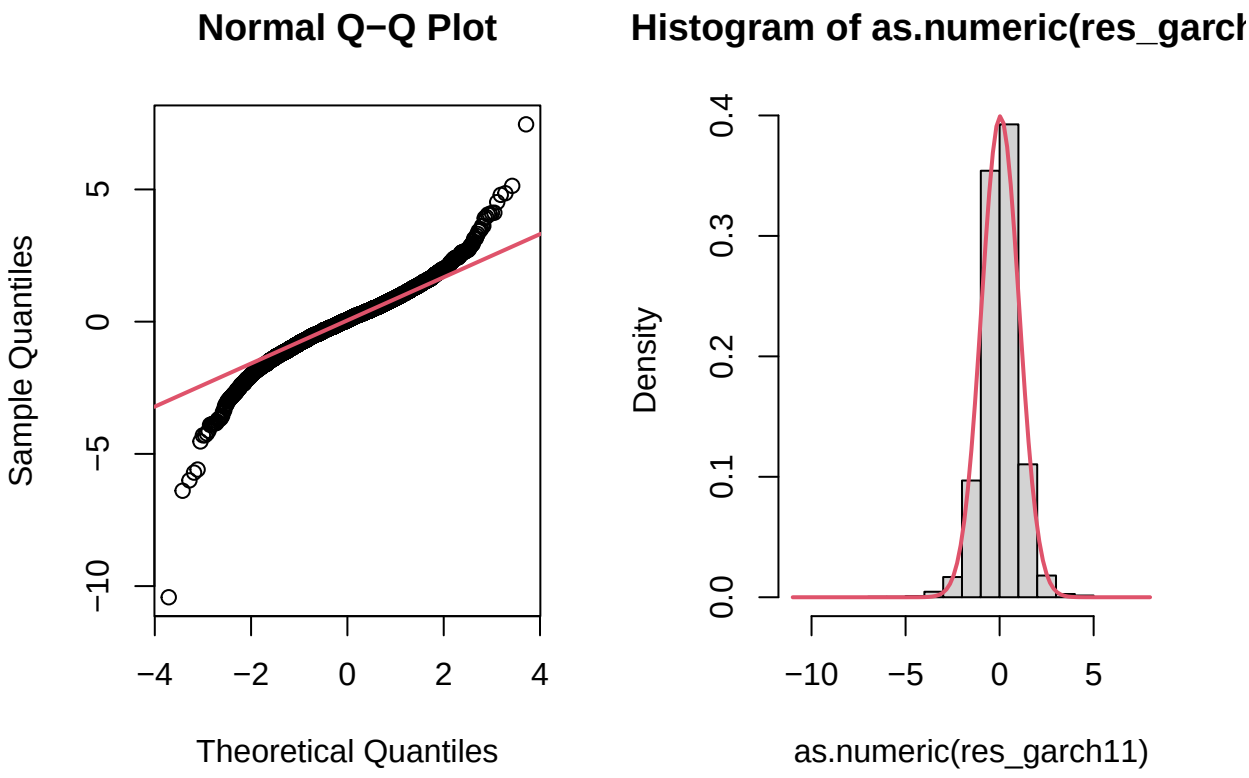
```r
out <- capture.output(summary(arch3))
start <- which(grepl("Jarque-Bera", out))
end   <- which(grepl("Shapiro-Wilk", out))
cat(out[start:end], sep = "\n")
```

```
##  Jarque-Bera Test   R    Chi^2  6806.2521997 0.00000000
##  Shapiro-Wilk Test  R    W         0.9585433 0.00000000
```

This is further confirmed by the Jarque-Bera and Shapiro-Wilk tests, which both reported a p-value of 0.000, rejecting the null-hypothesis of normality.

GARCH(1,1)

```r
par(mfrow = c(1, 2))
qqnorm(as.numeric(res_garch11))
qqline(as.numeric(res_garch11), col = 2, lwd = 2)
hist(as.numeric(res_garch11), probability = TRUE)
curve(dnorm(x, mean = mean(as.numeric(res_garch11)), sd = sd(as.numeric(res_garch11))), lwd = 2, col = 2
```




```
par(mfrow = c(1, 1))
```

```
stats <- basicStats(as.numeric(res_garch11))
stats[c("Skewness", "Kurtosis"), , drop = FALSE]
```

```
##          as.numeric.res_garch11
## Skewness              -0.408454
## Kurtosis               5.582239
```

Again, both the distribution plots and the skewness and kurtosis of the distribution suggest that the residuals are not normally distributed.

```
out <- capture.output(summary(garch11))
start <- which(grepl("Jarque-Bera", out))
end   <- which(grepl("Shapiro-Wilk", out))
cat(out[start:end], sep = "\n")
```

```
##  Jarque-Bera Test   R    Chi^2  6424.5365198 0.00000000
##  Shapiro-Wilk Test  R    W         0.9602602 0.00000000
```

This is further confirmed by the Jarque-Bera and Shapiro-Wilk tests, which both reported a p-value of 0.000, rejecting the null-hypothesis of normality.

Summary

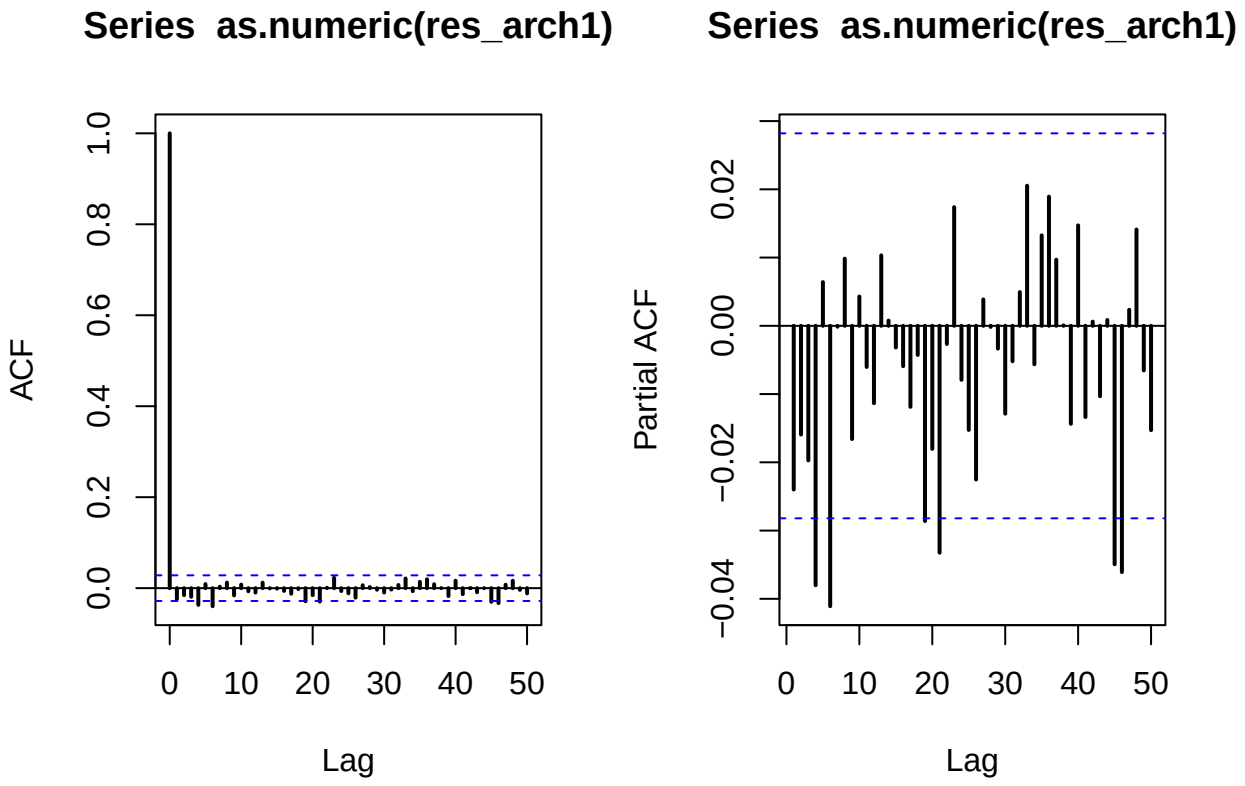
- Standardized residuals of $ARCH(1)$ are **not** normally distributed.
- Standardized residuals of $ARCH(3)$ are **not** normally distributed.
- Standardized residuals of $GARCH(1,1)$ are **not** normally distributed.

Testing dependence on standardized residuals

All of these models assume the standardized residuals to be independent to each other. To determine if they are independent, we plot their ACF and PACF and we analyze the results of the Ljung-Box test.

ARCH(1)

```
par(mfrow = c(1, 2))
acf(as.numeric(res_arch1), lwd = 2, lag.max = 50)
pacf(as.numeric(res_arch1), lwd = 2, lag.max = 50)
```




```
par(mfrow = c(1, 1))
```

Looking at the ACF and PACF, some lags cross the confidence intervals, so there could be some degree of dependence between the standardize residuals. The Ljung-Box test gives further insight about this.

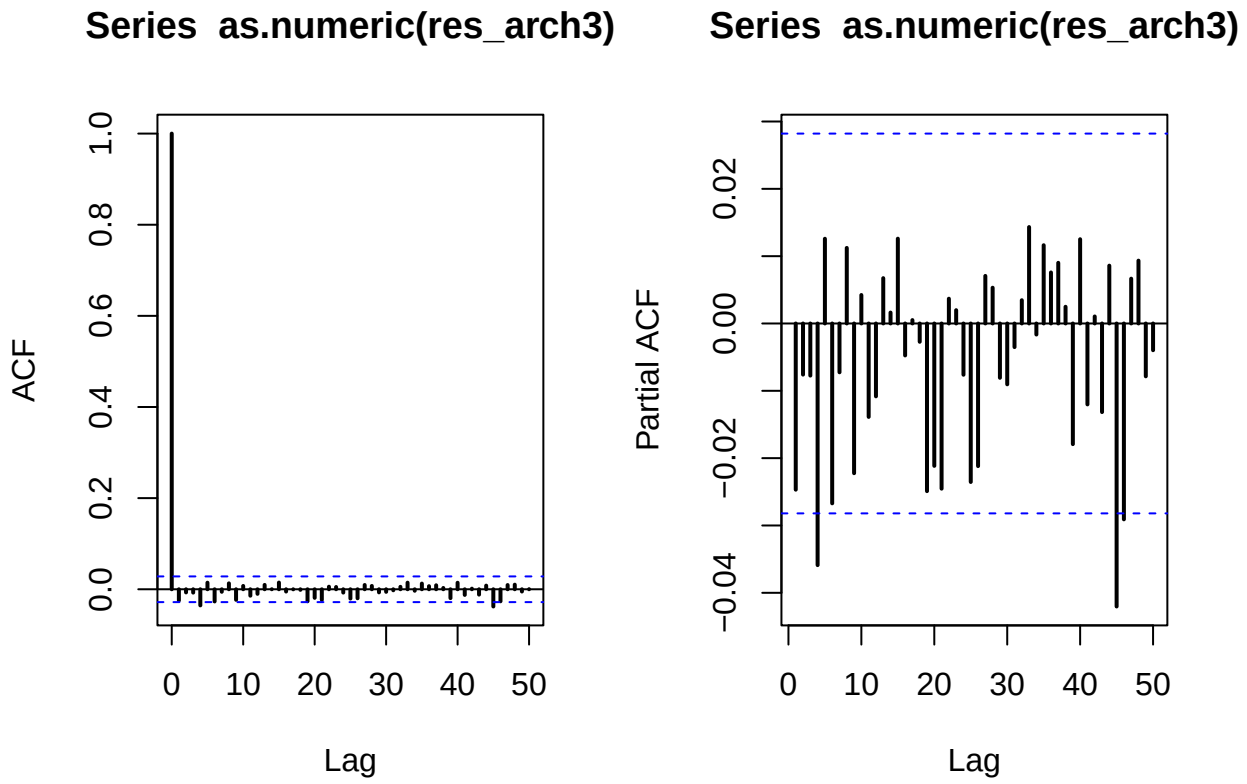
```r
out <- capture.output(summary(arch1))
start <- which(grepl("Ljung-Box", out))
end   <- which(grepl("Ljung-Box", out)) + 2
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test      R    Q(10)    22.879001 0.01120212
##  Ljung-Box Test      R    Q(15)    24.364302 0.05915965
##  Ljung-Box Test      R    Q(20)    30.424804 0.06325654
```

For lag 10, the p-value is smaller than 0.05, therefore rejecting the null hypothesis of independence between standardized residuals. However, the p-value is very close to 0.05, and higher lags don't reject the null. Although we can't formally conclude that the standardized residuals are independent, the degree of dependence may be very small.

ARCH(3)

```r
par(mfrow = c(1, 2))
acf(as.numeric(res_arch3), lwd = 2, lag.max = 50)
pacf(as.numeric(res_arch3), lwd = 2, lag.max = 50)
```



```r
par(mfrow = c(1, 1))
```

Very few lags cross the CI, which is acceptable since they are built at the 5% level. Hence, looking at the ACF and PACF, standardized residuals seem to be independent between each other.

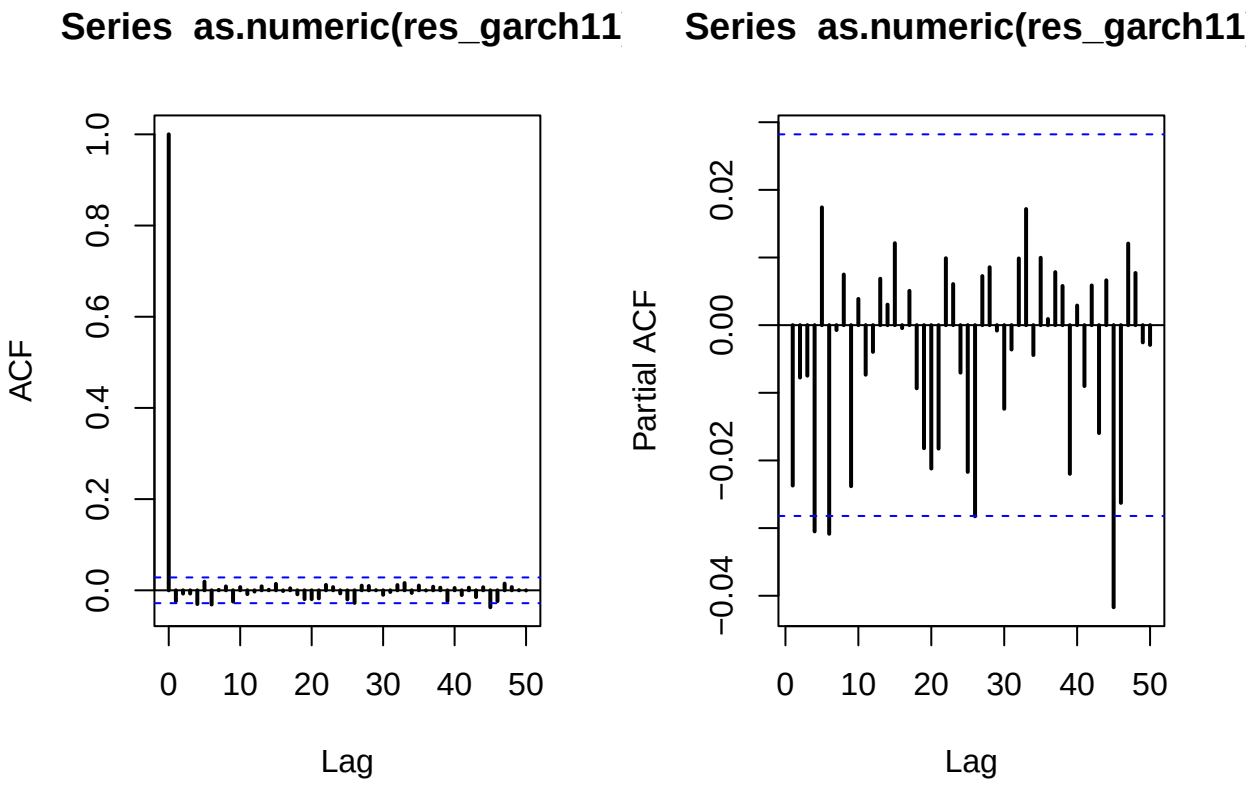
```r
out <- capture.output(summary(arch3))
start <- which(grepl("Ljung-Box", out))
end   <- which(grepl("Ljung-Box", out)) + 2
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test      R    Q(10)    18.3627127 0.04914485
##  Ljung-Box Test      R    Q(15)    21.4357256 0.12347583
##  Ljung-Box Test      R    Q(20)    26.7566074 0.14225158
```

Again, for lag 10 the p-value is smaller than 0.05, but very close. Therefore, although we can't formally conclude that the standardized residuals are independent, the degree of dependence may be very small.

GARCH(1,1)

```r
par(mfrow = c(1, 2))
acf(as.numeric(res_garch11), lwd = 2, lag.max = 50)
pacf(as.numeric(res_garch11), lwd = 2, lag.max = 50)
```



```r
par(mfrow = c(1, 1))
```

Again, very few lags cross the CI, which is acceptable since they are built at the 5% level. Hence, looking at the ACF and PACF, standardized residuals seem to be independent between each other.

```r
out <- capture.output(summary(garch11))
start <- which(grepl("Ljung-Box", out))
end   <- which(grepl("Ljung-Box", out)) + 2
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test     R    Q(10)    17.9065230 0.05656109
##  Ljung-Box Test     R    Q(15)    19.5665716 0.18919730
##  Ljung-Box Test     R    Q(20)    23.9170511 0.24603433
```

For every lag the p-value is greater than 0.05, failing to reject the null hypothesis of independence between standardized residuals. We can conclude that standardized residuals are independent.

Summary

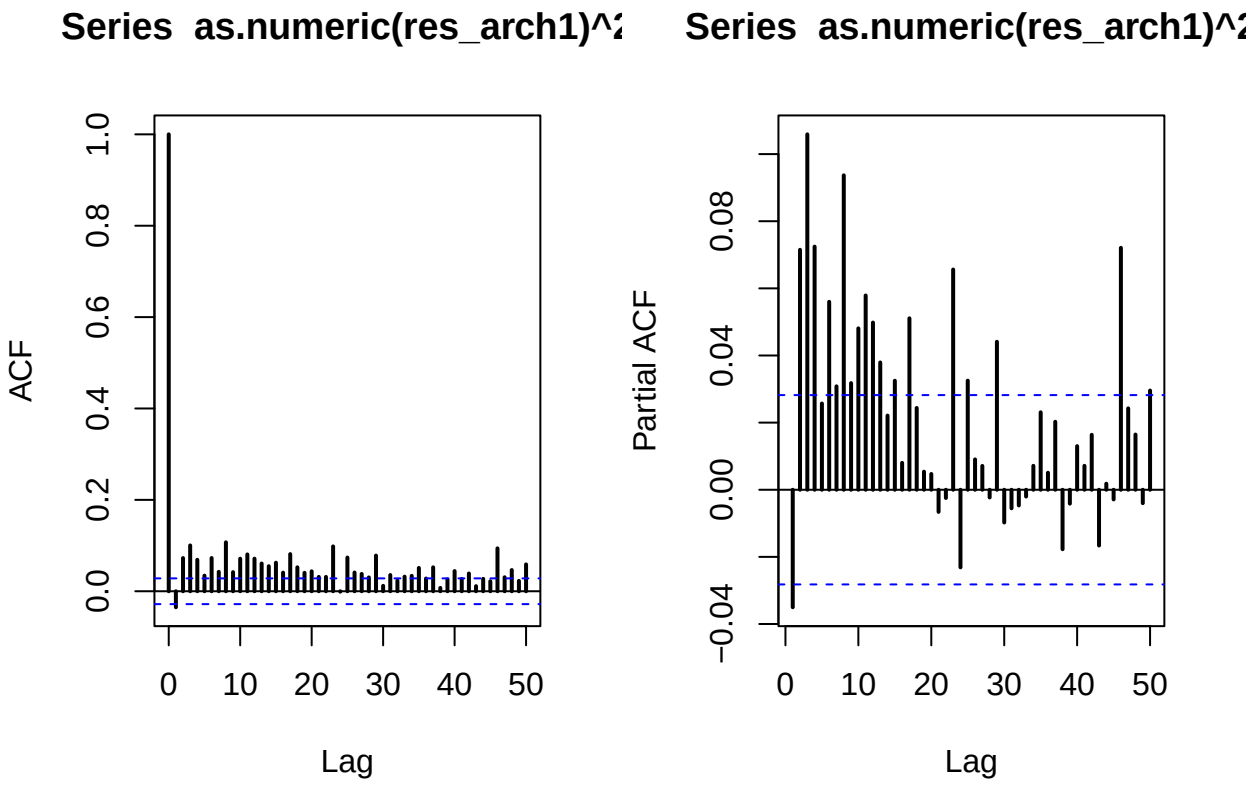
- Standardized residuals of $ARCH(1)$ are **not** independent, but there is only minor correlation.
- Standardized residuals of $ARCH(3)$ are **not** independent, but there is only minor correlation.
- Standardized residuals of $GARCH(1,1)$ are independent.

Testing heteroskedasticity of standardized residuals

If the models worked, they should have removed the heteroskedasticity of the time series. To test this, we look at the ACF and PACF of the squared standardized residuals and analyze the Ljung-Box and LM ARCH test results.

ARCH(1)

```r
par(mfrow = c(1, 2))
acf(as.numeric(res_arch1)^2, lwd = 2, lag.max = 50)
pacf(as.numeric(res_arch1)^2, lwd = 2, lag.max = 50)
```




```
par(mfrow = c(1, 1))
```

Looking at the ACF and PACF, a lot of lags cross the confidence intervals, so the squared standardized residuals are not independent, suggesting that heteroskedasticy is still present.

```
out <- capture.output(summary(arch1))
start <- which(grepl("Ljung-Box", out)) + 3
end   <- which(grepl("Ljung-Box", out)) + 5
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test      R^2  Q(10)    230.560729 0.00000000
##  Ljung-Box Test      R^2  Q(15)    337.567608 0.00000000
##  Ljung-Box Test      R^2  Q(20)    407.699730 0.00000000
```

The Ljung-Box test results confirm our previous conlusion. For each lag, the p-value is 0, rejecting the hypothesis of independence between squared standardized residuals. Hence, there is still presence of heteroskedasticity.

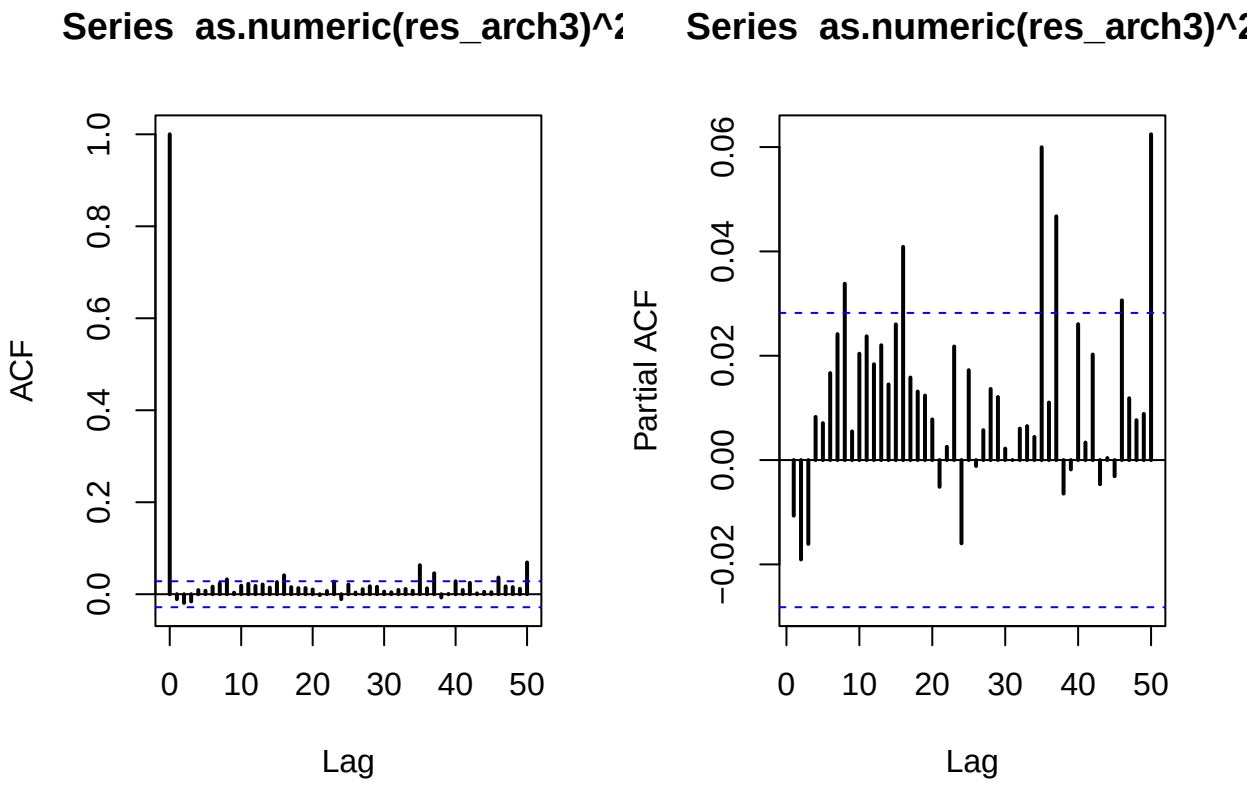
```
out <- capture.output(summary(arch1))
start <- which(grepl("LM", out))
end   <- which(grepl("LM", out))
cat(out[start:end], sep = "\n")
```

```
##  LM Arch Test        R     TR^2     212.249147 0.00000000
```

The LM ARCH test further confirms our findings, reporting a p-value of 0 and therefore rejecting the null hypothesis of homoskedastic residuals.

ARCH(3)

```
par(mfrow = c(1, 2))
acf(as.numeric(res_arch3)^2, lwd = 2, lag.max = 50)
pacf(as.numeric(res_arch3)^2, lwd = 2, lag.max = 50)
```




```
par(mfrow = c(1, 1))
```

Compared to the ACF and PACF of $ARCH(1)$, way less lags cross the confidence intervals, but still more than 5%. The Ljung-Box test will provide further information.

```
out <- capture.output(summary(arch3))
start <- which(grepl("Ljung-Box", out)) + 3
end   <- which(grepl("Ljung-Box", out)) + 5
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test      R^2  Q(10)    14.5501114 0.14934501
##  Ljung-Box Test      R^2  Q(15)    25.2745102 0.04637608
##  Ljung-Box Test      R^2  Q(20)    36.2678168 0.01430289
```

The Ljung-Box test rejects the null-hypothesis of independence at lags 15 and 20. Therefore, there still seems to be some presence of hetereoskedasticity.

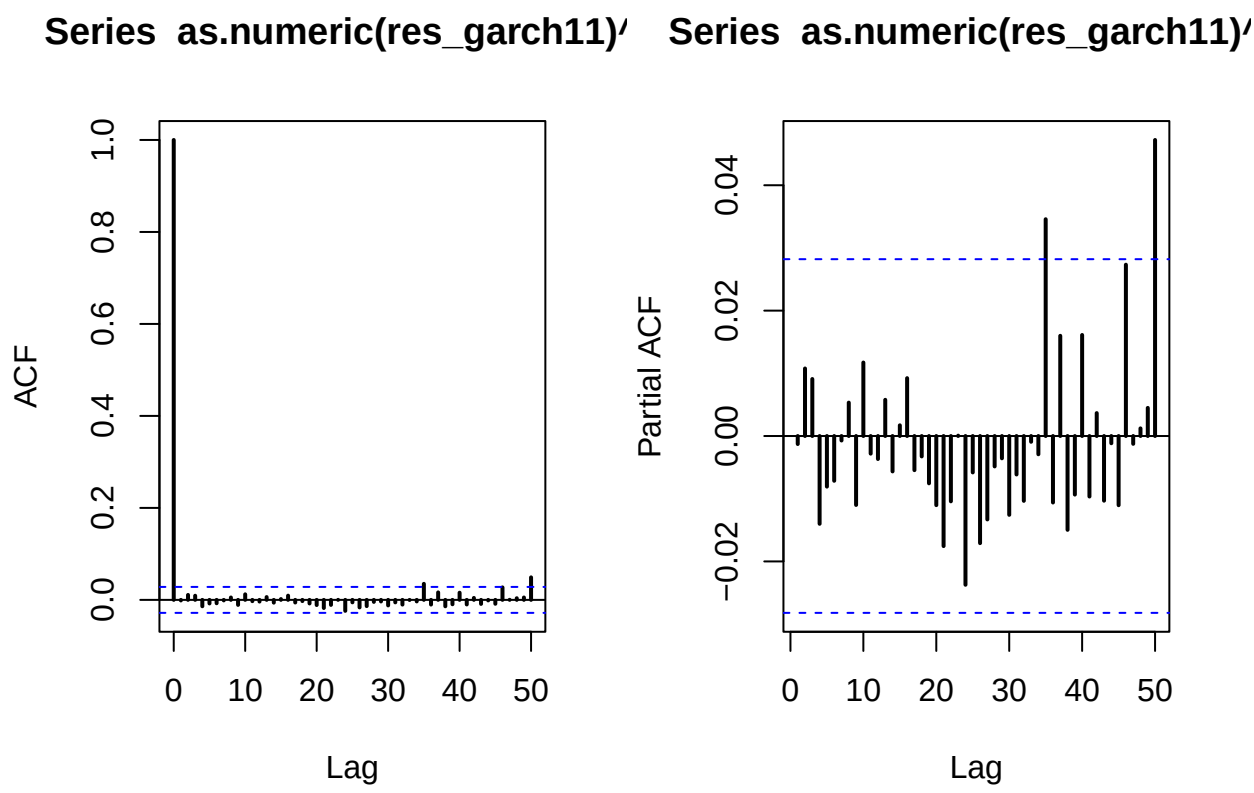
```
out <- capture.output(summary(arch3))
start <- which(grepl("LM", out))
end   <- which(grepl("LM", out))
cat(out[start:end], sep = "\n")
```

```
##  LM Arch Test       R    TR^2     19.5030538 0.07709004
```

The LM ARCH reports a p-value of around 0.07, not rejecting by a small margin the null hypothesis of homoskedasticity. This result is more or less in line with the one of the Ljung-Box test, with p-values also near the significance level.

GARCH(1,1)

```
par(mfrow = c(1, 2))
acf(as.numeric(res_garch11)^2, lwd = 2, lag.max = 50)
pacf(as.numeric(res_garch11)^2, lwd = 2, lag.max = 50)
```




```
par(mfrow = c(1, 1))
```

Now, even fewer lags than $ARCH(3)$ cross the confidence intervals, with a rate acceptable given the 5% level of the CIs. Therefore, squared standardized residuals seem to be independent, and $GARCH(1,1)$ may be able to explain most of the volatility in the data.

```
out <- capture.output(summary(garch11))
start <- which(grepl("Ljung-Box", out)) + 3
end   <- which(grepl("Ljung-Box", out)) + 5
cat(out[start:end], sep = "\n")
```

```
##  Ljung-Box Test     R^2  Q(10)     4.0154535 0.94664715
##  Ljung-Box Test     R^2  Q(15)     4.5460340 0.99532217
##  Ljung-Box Test     R^2  Q(20)     6.0072957 0.99888762
```

For every lag the p-value is close to 1, failing to reject the null hypothesis of independence. Hence, there is no presence of heteroskedasticity after fitting the GARCH(1,1) model.

```
out <- capture.output(summary(garch11))
start <- which(grepl("LM", out))
end   <- which(grepl("LM", out))
cat(out[start:end], sep = "\n")
```

```
##  LM Arch Test       R    TR^2      4.1348321 0.98087856
```

The LM ARCH test further supports our findings, reporting a p-value of almost 1 and therefore confirming that the residuals are homoskedastic.

Summary

- Squared standardized residuals of $ARCH(1)$ are **not** independent (heteroskedasticity is still present).
- Squared standardized residuals of $ARCH(3)$ are **not** independent (heteroskedasticity is still present).
- Squared standardized residuals of $GARCH(1,1)$ are independent (no heteroskedasticity).

Measures of fit

Finally, we compare model fit to the data.

For $ARCH(1)$:

```
out <- capture.output(summary(arch1))
start <- which(grepl("Information Criterion", out))
end <- which(grepl("Information Criterion", out)) + 2
cat(out[start:end], sep = "\n")
```

```
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.201308 -6.197280 -6.201308 -6.199894
```

For $ARCH(3)$:

```r
out <- capture.output(summary(arch3))
start <- which(grepl("Information Criterion", out))
end <- which(grepl("Information Criterion", out)) + 2
cat(out[start:end], sep = "\n")
```

```
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.290743 -6.284031 -6.290745 -6.288386
```

For $GARCH(1,1)$:

```r
out <- capture.output(summary(garch11))
start <- which(grepl("Information Criterion", out))
end <- which(grepl("Information Criterion", out)) + 2
cat(out[start:end], sep = "\n")
```

```
## Information Criterion Statistics:
##       AIC       BIC       SIC      HQIC
## -6.327881 -6.322511 -6.327882 -6.325996
```

AIC, BIC, SIC, and HQIC (note that the last two were never covered in class) are all lower for $GARCH(1,1)$, hence $GARCH(1,1)$ is the model that better fits the data.

Model decision

Since $GARCH(1,1)$ passes all of the tests besides the normality ones (while $ARCH(1)$ and $ARCH(3)$ do not) and it also provides the better fit to the data, we select $GARCH(1,1)$ as the best model between our choices to model the volatility of the time series.